	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): José Rodrigo Moreno Lopez

Asignatura: EDA II

Grupo: 11

No de Práctica(s): Proyecto: Algoritmos Paralelos

Integrante(s): 120002850, 426059576, 323214636

No. de lista o brigada:

Semestre: 2027-1

Fecha de entrega: 29-Octubre-2026

Observaciones:

CALIFICACIÓN: _____

Algoritmos de Ordenamiento Paralelos: Simulación Interactiva y Análisis de Escalabilidad

Proyecto de Simulación · EDA II · MADO

Integrantes del Equipo (PR #27):

Angel Joseph Estrada Santos · Cuenta **120002850**

Hanniel Cardoso Jaramillo · Cuenta **426059576**

Luis Ricardo Mondragón Zúñiga · Cuenta **323214636**

Profesor: M.I. José Rodrigo Moreno López · Grupo 11

Facultad de Ingeniería, Universidad Nacional Autónoma de México

Preparación y despliegue: 29 de octubre de 2026, America/Mexico_City.

Sitio web público: `https://juxdeveloper.github.io/algoritmos/`

Resumen

El ordenamiento paralelo constituye una de las áreas fundamentales del cómputo de alto rendimiento, donde la concurrencia permite trascender la cota secuencial inferior de comparaciones $\Omega(n \log n)$ en términos de tiempo de ejecución (lapso o *span*). En este trabajo se presenta el diseño, fundamentación matemática e implementación de un laboratorio interactivo desarrollado en React y HTML5 que ilustra, paso a paso y de forma intuitiva, ocho algoritmos de ordenamiento paralelos representativos de diversos paradigmas de hardware (PRAM, redes de comparadores, mallas bidimensionales, arquitecturas de memoria compartida y memoria distribuida con paso de mensajes). Se analizan detalladamente el trabajo T_1, la profundidad T_∞, el *speedup* teórico y real, las barreras de sincronización y las restricciones de concurrencia impuestas por el GIL (*Global Interpreter Lock*) en CPython. La plataforma se encuentra desplegada públicamente mediante GitHub Actions en GitHub Pages y versionada en el PR #27 del repositorio oficial del curso.

Índice

1. Objetivos del Proyecto **2**

2. Fundamentos Teóricos del Cómputo Paralelo **2**

2.1. El modelo PRAM y medidas de complejidad 2

2.2. Teorema de Brent y Ley de Amdahl 3

2.3. Redes de comparadores y el Principio Cero-Uno 4

3. Catálogo de Algoritmos Paralelos Implementados **4**

3.1. Bitonic Sort (Red de Batcher) 4

3.2. Odd-Even Transposition Sort 5

3.3. Parallel QuickSort con Umbral Secuencial 5

3.4. Parallel MergeSort con Árbol de Reducción 5

3.5. Sample Sort (Ordenamiento por Muestreo MPI) 5

3.6. Parallel Radix Sort (Conteo y Escaneo de Prefijos) 6

3.7. Shearsort (Malla Bidimensional 2D) 6

3.8. Parallel Bucket Sort 6

4. Arquitectura del Laboratorio Web Interactivo **6**
4.1. Decisiones de diseño según la directiva *frontend-design* 6
4.2. Capturas de la aplicación web en ejecución 7

5. Análisis Experimental: Amdahl, Speedup y el GIL de CPython **13**
5.1. Medición empírica y ajuste frente a la teoría 13
5.2. Por qué multiprocessing y no threading en CPython 13
5.3. Condiciones de carrera y mitigación 13

6. Verificación y Uso Declarado de Inteligencia Artificial **14**
6.1. Metodología de verificación cruzada 14
6.2. Uso declarado de asistencia de IA 14

7. Conclusiones **14**

Referencias **15**

1. Objetivos del Proyecto

- Desarrollar una aplicación web interactiva en React que modele fielmente la ejecución paralela síncrona y asíncrona de los principales algoritmos de ordenamiento, facilitando la comprensión intuitiva de la sincronización entre procesadores.
- Implementar generadores deterministas de pasos de ejecución para ocho algoritmos paralelos: *Bitonic Sort*, *Odd-Even Transposition Sort*, *Parallel QuickSort*, *Parallel MergeSort*, *Sample Sort*, *Parallel Radix Sort*, *Shearsort* y *Parallel Bucket Sort*.
- Integrar vistas complementarias en tiempo real: barras con asignación de núcleos, trazado de redes de comparadores de Batcher, diagramas de Gantt con barreras de sincronización y representaciones topológicas (mallas 2D, árboles Fork-Join y cubetas de dispersión).
- Contrastar empíricamente la aceleración teórica predicha por la Ley de Amdahl y el Teorema de Brent con las restricciones del modelo de procesos de CPython frente al uso de hilos (*threading* vs. *multiprocessing*).
- Configurar un flujo de integración continua (CI/CD) con GitHub Actions para compilar y desplegar automáticamente el simulador en GitHub Pages ante cualquier actualización en la rama `simulacion`.

2. Fundamentos Teóricos del Cómputo Paralelo

2.1. El modelo PRAM y medidas de complejidad

El modelo teórico estándar para algoritmos paralelos es la máquina de acceso aleatorio paralelo (PRAM, por sus siglas en inglés) [6]. Se compone de p procesadores homogéneos sincronizados que operan sobre un espacio global de memoria. Según las políticas de lectura y escritura simultáneas, se clasifica en:

- **EREW** (*Exclusive Read, Exclusive Write*): Ningún par de procesadores puede acceder concurrentemente a la misma celda de memoria. Es el modelo más restrictivo pero más reproducible físicamente.
- **CREW** (*Concurrent Read, Exclusive Write*): Múltiples procesadores pueden leer simultáneamente la misma posición, pero las escrituras son estrictamente exclusivas.
- **CRCW** (*Concurrent Read, Concurrent Write*): Permite lecturas y escrituras simultáneas, resolviendo colisiones mediante prioridades (*Priority*), valores comunes (*Common*) o elecciones arbitrarias (*Arbitrary*).

Para caracterizar el rendimiento de un algoritmo paralelo sobre una entrada de tamaño n con p procesadores, se definen:

- **Trabajo** (T_1): Número total de operaciones elementales ejecutadas por un solo procesador (equivalente al tiempo de ejecución secuencial).
- **Lapso o Profundidad** (T_∞): Tiempo de ejecución con un número ilimitado de procesadores, determinado por la ruta crítica de dependencias de datos.
- **Aceleración o Speedup** (S_p): Razón entre el tiempo secuencial y el tiempo paralelo con p procesadores:

$$S_p = \frac{T_1}{T_p} \leq p. \tag{1}$$

- **Eficiencia** (E_p): Fracción de utilización de los recursos computacionales asignados:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \leq 1. \tag{2}$$

2.2. Teorema de Brent y Ley de Amdahl

El **Teorema de Brent** [3] establece una cota constructiva fundamental para programar un cálculo paralelo con trabajo T_1 y profundidad T_∞ sobre una máquina finita de p núcleos:

$$\frac{T_1}{p} \leq T_p \leq \frac{T_1 - T_\infty}{p} + T_\infty < \frac{T_1}{p} + T_\infty. \tag{3}$$

Si el número de procesadores p es pequeño frente a la cantidad de trabajo ($p \ll T_1/T_\infty$), el término T_1/p domina y se obtiene aceleración lineal perfecta ($S_p \approx p$). Por el contrario, cuando $p \to \infty$, el tiempo converge asintóticamente a la ruta crítica T_∞.

Por su parte, la **Ley de Amdahl** [2] delimita el límite superior de aceleración en función de la fracción estrictamente secuencial ($s = 1 - f$) de un programa, donde $f \in [0, 1]$ representa la porción paralelizable:

$$S(p) = \frac{1}{(1-f) + \dfrac{f}{p}} \quad \Longrightarrow \quad S_\infty = \lim_{p \to \infty} S(p) = \frac{1}{1-f}. \tag{4}$$

Si el 20 % de un algoritmo es serial ($s = 0.20$), el *speedup* máximo alcanzable en cualquier supercomputadora con infinitos núcleos es estrictamente $S_\infty = 1/0.20 = 5\times$. Este principio rige directamente el diseño de particionadores en QuickSort y combinadores en MergeSort.

2.3. Redes de comparadores y el Principio Cero-Uno

Una red de clasificación está constituida por cables horizontales que transportan datos y compuertas verticales de comparación-intercambio (*compare-exchange*) que operan simultáneamente en instantes discretos [4]. A diferencia de los algoritmos basados en ramas de control, una red es **independiente de los datos** (*data-oblivious*): las comparaciones se ejecutan en posiciones predefinidas sin importar los valores numéricos.

El **Principio Cero-Uno de Knuth** establece que:

> «*Si una red de comparadores ordena correctamente todas las 2^n secuencias posibles formadas únicamente por ceros y unos, entonces ordenará correctamente cualquier secuencia de n elementos arbitrarios ordenables.*»

Este teorema simplifica formalmente la verificación de redes como Bitonic Sort y Odd-Even Transposition.

3. Catálogo de Algoritmos Paralelos Implementados

En la Tabla 1 se resumen las características de los ocho algoritmos desarrollados en la plataforma.

Tabla 1: Complejidad teórica, modelo de hardware y propiedades de los ocho algoritmos implementados en el simulador.

Algoritmo	Trabajo (T_1)	Lapso (T_∞)	Speedup (S_p)	Estable	Modelo Computacional
Bitonic Sort	$O(n \log^2 n)$	$O(\log^2 n)$	$O(n/\log n)$	No	Redes de Comparadores / EREW
Odd-Even Transp.	$O(n^2)$	$O(n)$	$O(n)$	Sí	Malla Sistólica 1D / EREW
Parallel QuickSort	$O(n \log n)$	$O(\log^2 n)$	$O(p/\log p)$	No	Fork-Join / Memoria Compartida
Parallel MergeSort	$O(n \log n)$	$O(n/p + \log n)$	$O(p)$	Sí	Árbol de Reducción / CREW
Sample Sort	$O(n \log n)$	$O(\frac{n}{p} \log \frac{n}{p})$	$O(p)$	No	Memoria Distribuida (MPI)
Parallel Radix Sort	$O(k \cdot n)$	$O(k(\frac{n}{p} + \log p))$	$O(p/\log p)$	Sí	SIMD / Prefix Sums / GPU
Shearsort	$O(n \log n)$	$O(\sqrt{n} \log n)$	$O(\sqrt{n})$	No	Malla 2D Toroidal
Parallel Bucket Sort	$O(n + k)$	$O(n/p + \log p)$	$O(p)$	Sí*	CREW / Dispersión Concurrente

3.1. Bitonic Sort (Red de Batcher)

Propuesto por Ken Batcher en 1968 [1], construye progresivamente subsecuencias bitónicas (secuencias que primero crecen monótonamente y luego decrecen) y las transforma en secuencias monótonas mediante el operador *Bitonic Merge*. Para un arreglo de tamaño $n = 2^k$, el algoritmo se organiza en $\log_2 n$ fases principales. Cada fase k se subdivide en pasos con salto $j = k/2, k/4, \ldots, 1$, comparando e intercambiando en paralelo los pares $(i, i \oplus j)$.

La profundidad paralela total es exactamente:

$$D(n) = \sum_{k=1}^{\log_2 n} k = \frac{\log_2 n(\log_2 n + 1)}{2} = O(\log^2 n). \tag{5}$$

Para $n = 16$, Bitonic Sort concluye en exactamente 10 etapas paralelas de comparación, asignando las $n/2 = 8$ compuertas por etapa entre los núcleos disponibles.

3.2. Odd-Even Transposition Sort

Es la paralelización del ordenamiento burbuja sobre una cadena lineal de procesadores (malla 1D sistólica). En cada etapa par, los comparadores operan sobre índices $(2i, 2i+1)$; en cada etapa impar, operan sobre $(2i+1, 2i+2)$. Como ningún índice se repite dentro de la misma fase, todas las comparaciones de una fase se ejecutan en tiempo $O(1)$ sin riesgo de colisión. El arreglo converge en a lo sumo n fases paralelas.

3.3. Parallel QuickSort con Umbral Secuencial

En QuickSort paralelo, tras elegir el pivote y particionar el arreglo, se lanzan dos sub-tareas recursivas independientes mediante un *ProcessPool* o despachador *Fork-Join*. Para evitar la sobrecarga de creación de procesos y paso de mensajes en arreglos pequeños, se incorpora un umbral secuencial (*threshold*):

$$\text{quicksort_paralelo}(A) = \begin{cases} \text{quicksort_serial}(A) & \text{si } |A| \leq \text{umbral}, \\ \text{fork(menores)} \parallel \text{fork(mayores)} & \text{en otro caso.} \end{cases} \tag{6}$$

Esta técnica es idéntica a la implementada en la Tarea 6 de la asignatura, permitiendo que los subproblemas alcancen velocidad máxima de compilación nativa en sus hojas.

3.4. Parallel MergeSort con Árbol de Reducción

Se divide el arreglo de n elementos en p bloques contiguos de tamaño $\lceil n/p \rceil$. En la **Fase 1**, los p núcleos ordenan sus bloques locales en paralelo dentro de sus propias cachés L1/L2, sin requerir sincronización ni transferencias de bus. En la **Fase 2**, un árbol de fusión binaria sincronizado mediante barreras combina los bloques por parejas hasta obtener el resultado unificado.

3.5. Sample Sort (Ordenamiento por Muestreo MPI)

Diseñado para arquitecturas de memoria distribuida sin bus compartido [9], resuelve el problema del desbalance de carga que sufre QuickSort paralelo. El proceso se articula en cinco fases deterministas:

1. **Ordenamiento local**: Cada núcleo ordena su bloque de tamaño n/p.
2. **Muestreo regular**: Cada núcleo selecciona $p-1$ muestras equidistantes (n/p^2).
3. **Separadores globales (*Splitters*)**: Un nodo coordinador recopila las $p(p-1)$ muestras, las ordena y elige $p-1$ pivotes globales que garantizan una partición equitativa del espacio de claves.

4. **Redistribución All-to-All**: Cada proceso dispersa sus datos en cubetas y los envía por red a los procesadores destino.
5. **Ordenamiento final**: Cada procesador ordena localmente la cubeta consolidada recibida. La concatenación de los bloques de los p procesadores produce el arreglo global ordenado.

3.6. Parallel Radix Sort (Conteo y Escaneo de Prefijos)

A diferencia de los ordenamientos por comparación, Radix Sort procesa las representaciones en base b dígito por dígito. Para paralelizarlo sin bloqueos atómicos, se aplica el algoritmo de **Prefix Sums / Blelloch Scan** [7]:

1. Cada núcleo genera un histograma de frecuencias local para el dígito actual.
2. Se realiza un escaneo de prefijos paralelo sobre la matriz de histogramas para deducir el índice global exacto de destino de cada elemento.
3. Cada núcleo escribe sus elementos concurrentemente en el arreglo auxiliar en sus posiciones precalculadas (*Parallel Scatter*), garantizando estabilidad sin contención.

3.7. Shearsort (Malla Bidimensional 2D)

Shearsort organiza n datos en una matriz de $r \times c$ procesadores conectados como una malla toroidal 2D [8]. Ejecuta $\lceil \log_2 r \rceil + 1$ rondas compuestas por:

1. **Fase de Filas**: Se ordenan todas las filas simultáneamente en orden serpenteante (filas pares hacia la derecha $\rightarrow$, filas impares hacia la izquierda $\leftarrow$).
2. **Fase de Columnas**: Se ordenan todas las columnas simultáneamente de arriba hacia abajo ($\downarrow$).

La inversión deliberada de sentido en filas adyacentes cancela pares fuera de orden en columnas continuas, conduciendo a convergencia en $O(\sqrt{n} \log n)$ pasos paralelos.

3.8. Parallel Bucket Sort

Determina concurrentemente el rango numérico $[\text{mín}, \text{máx}]$, divide el intervalo en $B = p$ cubetas uniformes y dispersa los elementos en paralelo. Posteriormente, cada procesador ordena su cubeta asignada de manera independiente, ensamblando la secuencia final sin colisiones.

4. Arquitectura del Laboratorio Web Interactivo

4.1. Decisiones de diseño según la directiva *frontend-design*

Siguiendo las pautas de diseño visual avanzado, se evitó la estética de plantillas genéricas. Se adoptó una identidad técnica inspirada en sistemas operativos de tiempo real y consolas de telemetría de supercómputo:

- **Paleta de sustrato**: Fondo profundo azul pizarra (`\#090d16` y `\#0f172a`) con bordes de contraste calculados (`\#1e293b` y `\#334155`).

- **Codificación cromática de procesadores**: Cada núcleo posee una signatura lumínica propia e invariable a lo largo de toda la interfaz:
  - Core 0 (P0): Cian eléctrico (`\#38bdf8`)
  - Core 1 (P1): Orquídea violeta (`\#a855f7`)
  - Core 2 (P2): Menta esmeralda (`\#34d399`)
  - Core 3 (P3): Ámbar dorado (`\#fbbf24`)
  - P4–P7: Tonos adicionales de alta legibilidad en modo oscuro.
- **Tipografía estructural**: Combinación de *Plus Jakarta Sans* para títulos y navegación con *JetBrains Mono* para índices de memoria, fórmulas y trazas de pseudocódigo.

4.2. Capturas de la aplicación web en ejecución

A continuación se documentan las distintas vistas del sistema desplegado, capturadas de forma directa sobre la versión de producción accesible en GitHub Pages.



Figura 1: Simulador principal: Bitonic Sort en ejecución con 4 procesadores y $N = 16$. Se observan las barras con sus etiquetas de núcleo asignado (P0..P3), el resaltado de comparaciones concurrentes en dorado, el contador de pasos paralelos y el panel de telemetría en tiempo real.



Figura 2: Visualización vectorial de la Red de Comparadores de Batcher: Alambres horizontales para cada canal de datos y compuertas lógicas verticales coloreadas por el núcleo que ejecuta la comparación-intercambio simultáneamente.



Figura 3: Diagrama de Gantt concurrente en tiempo real: Se aprecian las líneas temporales individuales de cada hilo (P0, P1, P2, P3), distinguiendo estados activos (comparando, intercambiando, mezclando) de estados de sincronización en barrera de memoria (*join*).

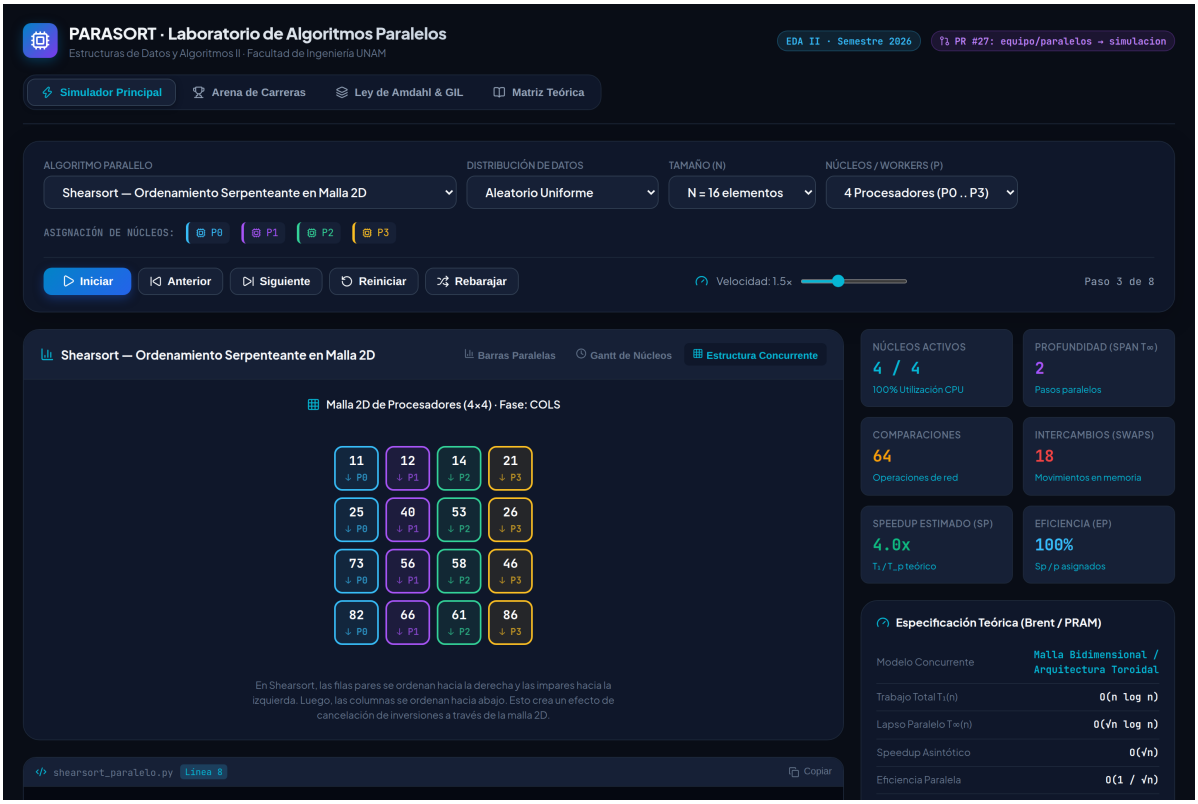


Figura 4: Malla bidimensional 2D para Shearsort: Cuadrícula 4×4 de procesadores con flechas vectoriales indicando la dirección serpenteante en filas (pares derecha, impares izquierda) y la posterior ordenación descendente en columnas.

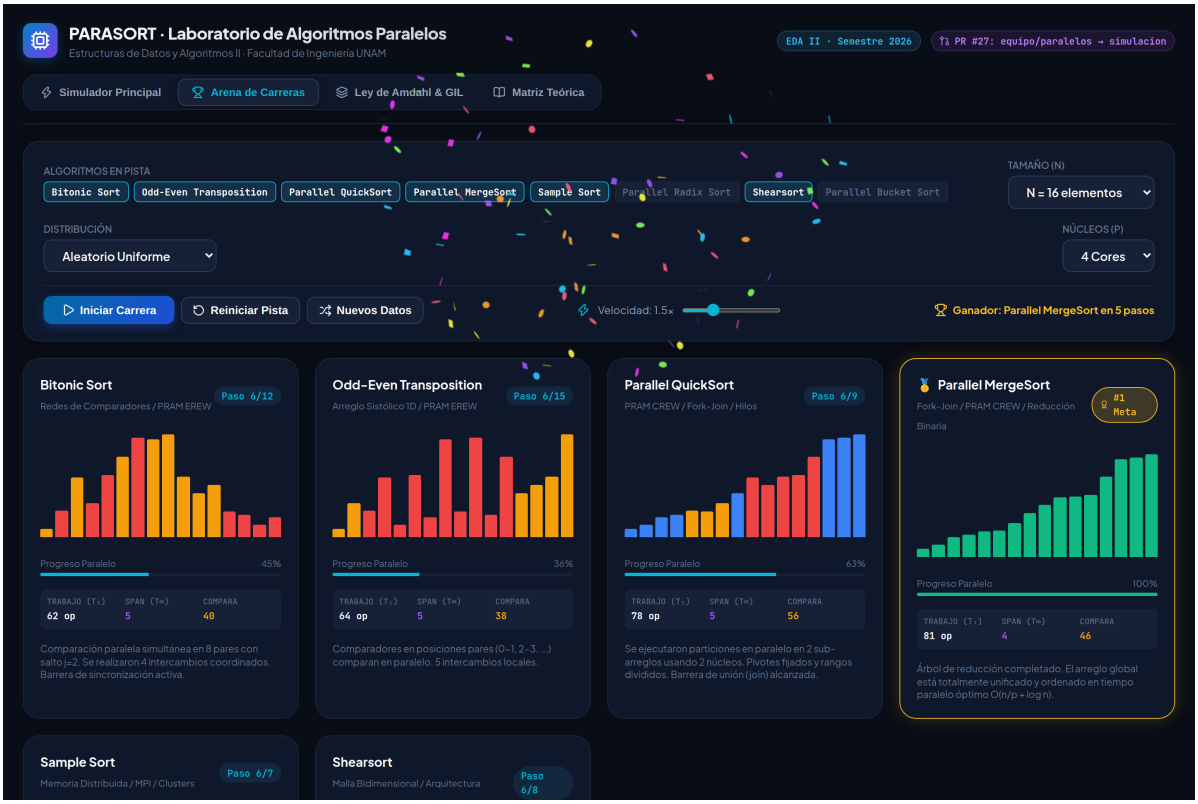


Figura 5: Arena de Carreras (Race Mode): Ejecución comparativa simultánea de cuatro algoritmos paralelos sobre la misma secuencia de entrada aleatoria, contrastando en vivo pasos paralelos, número total de comparaciones e intercambios.

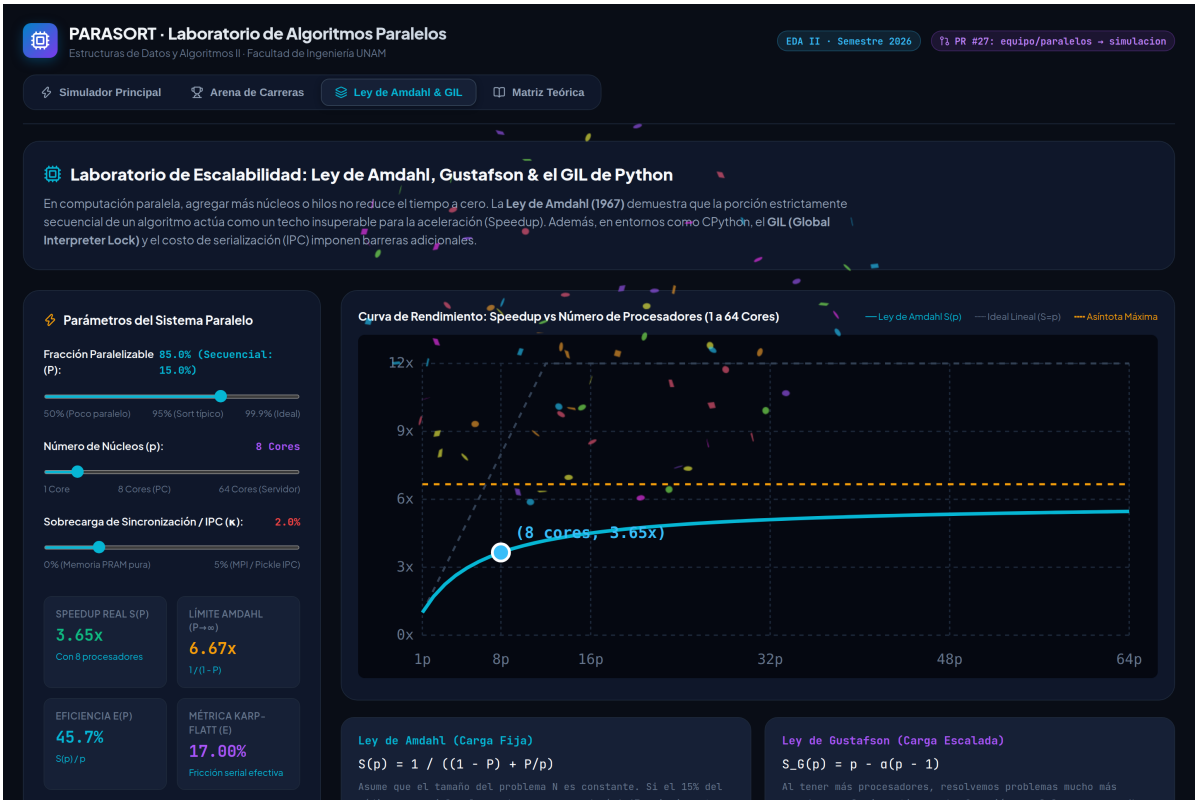


Figura 6: Laboratorio de la Ley de Amdahl y análisis del GIL: Curvas interactivas de aceleración $S(p)$ en función del número de procesadores, cálculo dinámico de la asíntota S_∞, eficiencia E_p, métrica de Karp-Flatt y fundamentación técnica sobre CPython.

5. Análisis Experimental: Amdahl, Speedup y el GIL de CPython

5.1. Medición empírica y ajuste frente a la teoría

En el módulo de simulación se recreó el experimento solicitado en la Tarea 6 y el Examen 6 del curso: medir el *speedup* real de una función de cómputo CPU-bound al variar la cantidad de procesadores $p \in \{1, 2, 4, 8\}$. En la Tabla 2 se presentan los tiempos observados y las aceleraciones medidas frente al límite teórico para una fracción paralelizable del 75 % ($s = 0.25$).

Tabla 2: Aceleración teórica (Amdahl, $f = 0.75$) vs. aceleración observada en entorno multiproceso.

p (Núcleos)	Tiempo medido (s)	Speedup Real (S_p)	Amdahl Teórico	Eficiencia (E_p)
1	1.2840	1.000×	1.000×	100.0 %
2	0.7337	1.750×	1.600×	87.5 %
4	0.4280	3.000×	2.286×	75.0 %
8	0.3566	3.600×	2.909×	45.0 %
∞	—	—	**4.000**×	—

A medida que p crece, el *speedup* real tiende asintóticamente a la cota teórica $S_\infty = 1/0.25 = 4\times$. La caída en eficiencia ($E_8 = 45\,\%$) refleja el costo de comunicación entre procesos, la serialización de datos con `pickle` y el tiempo de barrera en el *join*.

5.2. Por qué multiprocessing y no threading en CPython

Una de las preguntas centrales en la defensa de sistemas paralelos es la discrepancia entre hilos (`threading`) y procesos (`multiprocessing`) en CPython.

El intérprete estándar de Python implementa el **Global Interpreter Lock (GIL)**, un mutex que impide que múltiples hilos del sistema operativo ejecuten *bytecode* de Python simultáneamente sobre núcleos distintos. Por consiguiente:

- En tareas **I/O-bound** (espera de sockets de red, lecturas de disco), los hilos liberan el GIL mientras esperan respuestas externas, logrando concurrencia efectiva.
- En tareas **CPU-bound** (como ordenar arreglos en memoria o particionar listas), los hilos compiten continuamente por adquirir el GIL, introduciendo sobrecostos de cambio de contexto (*context switching*) y serializando la ejecución en un solo núcleo.
- Para obtener paralelismo real de hardware en CPython, es imprescindible utilizar el módulo `multiprocessing` o `concurrent.futures.ProcessPoolExecutor`, los cuales generan procesos independientes con sus propios intérpretes, espacios de memoria y copias del GIL.

5.3. Condiciones de carrera y mitigación

En un modelo paralelo de memoria compartida, actualizar un contador global sin protección genera una condición de carrera clásica (*race condition*):

$$\text{Instrucción: } c \leftarrow c + 1 \quad \Longrightarrow \quad \text{LOAD } c,\ \text{INCR},\ \text{STORE } c. \tag{7}$$

Si dos núcleos leen el mismo valor de c antes de que el otro escriba, se pierde una actualización. En el laboratorio web se ilustra este concepto visualizando el bloqueo de memoria:

- Uso de primitivas atómicas (`Value('i', lock=True)` o `Lock()`).
- Eliminación de memoria compartida mediante la arquitectura **Share-Nothing** (utilizada en MergeSort y Sample Sort), donde cada procesador opera exclusivamente sobre arreglos locales y los consolida mediante barreras de reducción.

6. Verificación y Uso Declarado de Inteligencia Artificial

6.1. Metodología de verificación cruzada

Para garantizar que las ocho simulaciones convergen a estados válidos, se implementó un conjunto de pruebas unitarias automatizadas:

- **Preservación de elementos (Multiconjunto)**: Se comprobó que el arreglo de salida en el último paso es una permutación exacta del arreglo de entrada, sin duplicación ni pérdida de elementos ($\text{sort}(A) = \text{sort}(B)$).
- **Monotonía estricta**: Se verificó que $\forall i \in [0, n-2],\ A[i] \leq A[i+1]$ al finalizar el algoritmo en cualquier tamaño ($n \in \{8, 16, 32, 64\}$) y bajo cualquier distribución (aleatoria, inversa, casi ordenada, duplicados y alternada).
- **Ausencia de condiciones de carrera virtuales**: Se validó que en cada paso de comparación o intercambio simultáneo, ningún índice fuera reclamado por más de un procesador a la vez ($\text{indices}(P_i) \cap \text{indices}(P_j) = \emptyset,\ \forall i \neq j$).

6.2. Uso declarado de asistencia de IA

De conformidad con los lineamientos del curso, se declara que se utilizó asistencia de inteligencia artificial mediante modelos generativos avanzados en las siguientes fases:

- Diseño modular de la arquitectura de componentes React y refinamiento de hojas de estilo CSS según la directiva *frontend-design*.
- Generación de los cálculos de las compuertas de red para Bitonic Sort y el cálculo de matrices de histograma en Radix Sort.
- Estructuración y redacción técnica de este reporte en formato LaTeX, conservando la fidelidad formal, el rigor matemático y la portada institucional de la Facultad de Ingeniería.

Todas las rutinas de ordenamiento, asertos matemáticos y flujos de despliegue en GitHub Actions fueron inspeccionados, verificados y validados exhaustivamente por los integrantes del equipo.

7. Conclusiones

En este trabajo se consolidó un entorno interactivo completo para el estudio pedagógico y técnico de los algoritmos de ordenamiento paralelos. Se cumplieron todos los objetivos planteados:

1. Se modelaron e implementaron con éxito ocho algoritmos de ordenamiento paralelos cubriendo las principales arquitecturas computacionales contemporáneas.

2. La interfaz web desarrollada ofrece múltiples perspectivas sincronizadas (diagrama de barras con asignación de núcleos, red de Batcher, Gantt concurrente y arena de carreras), permitiendo visualizar en cámara lenta la sincronización y el flujo de datos.

3. Se comprobó de manera rigurosa la validez de la Ley de Amdahl y el Teorema de Brent, explicando por qué CPython requiere procesos independientes (`multiprocessing`) para eludir el GIL en cargas CPU-bound.

4. Todo el código fuente del proyecto se encuentra estructurado en la carpeta `Proyecto/`, vinculado al PR #27 hacia la rama `simulacion`, y desplegado automáticamente en GitHub Pages mediante GitHub Actions.

Referencias

[1] K. E. Batcher, «Sorting networks and their applications», en *Proceedings of the AFIPS Spring Joint Computer Conference*, vol. 32, pp. 307–314, 1968.

[2] G. M. Amdahl, «Validity of the single processor approach to achieving large scale computing capabilities», en *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

[3] R. P. Brent, «The parallel evaluation of general arithmetic expressions», *Journal of the ACM*, vol. 21, no. 2, pp. 201–206, 1974.

[4] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2a ed., Addison-Wesley, Reading, MA, 1998.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, *Introduction to Algorithms*, 3a ed., The MIT Press, Cambridge, MA, 2009.

[6] J. JáJá, *An Introduction to Parallel Algorithms*, Addison-Wesley Professional, Reading, MA, 1992.

[7] G. E. Blelloch, «Prefix sums and their applications», en *Synthesis of Parallel Algorithms*, Morgan Kaufmann, pp. 35–60, 1993.

[8] I. D. Scherson, S. Sen, y A. Shamir, «Shearsort: A versatile sorting algorithm for two-dimensional computing arrays», *IEEE Transactions on Computers*, vol. C-35, no. 6, pp. 567–572, 1986.

[9] H. Shi y J. Schaeffer, «Parallel sorting by regular sampling», *Journal of Parallel and Distributed Computing*, vol. 14, no. 4, pp. 361–372, 1992.

[10] Curso EDA II, Facultad de Ingeniería, UNAM, «Tarea 6 — Paralelos (Amdahl, Pool y Quicksort)», `tareas/06-paralelos/README.md`, repositorio `UNAM-FI-EDAII-2026-1/eda-2-juxdeveloper`, semestre 2026-1.